UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

FACULTAD DE INGENIERÍA DE SISTEMAS E INFORMÁTICA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS E INFORMÁTICA

Actividad grupal - Semana 04

Módulo de auditoría y monitoreo para el sistema "Campus 360"

INTEGRANTES DEL GRUPO N° 09

- Ccente Alfonso Renzo Piero (23200152)
- Chávez Chacón Alex Junior (23200155)
- Diaz Sobrino Yuzo (23200167)
- Muñoz Aliaga Bryan Stephano (23200038)
- Sanchez Osorio Miguel Angel (23200210)
- Barrientos Torres Jose Paolo (23200145)

ASIGNATURA

Diseño de Sistemas de Información

DOCENTE

Andrade Mogollon, Teodoro Manuel

Lima, Perú

2026

ÍNDICE

1. Descripción del problema.....3
2. Justificación.....3
3. Objetivo..... 4
4. Alcance de negocio.....4
5. Actores del sistema..... 4
6. Lista de requisitos.....5
6.1. Requisitos Funcionales.....5
6.2. Requisitos No Funcionales..... 6
7. Backlog.....7
8. Reglas de negocio.....9
9. Estilo arquitectónico.....10
10. Arquitectura 4 +1.....11
11. Diseño de Interfaces del Módulo de Auditoría.....19
12. Prototipo Mínimo Funcional (MVP).....22

1. Descripción del problema

En la actualidad, los sistemas universitarios gestionan múltiples trámites y solicitudes que pasan por distintas áreas y personas, como estudiantes, personal administrativo y jefes de área. Sin embargo, muchas veces no existe un mecanismo claro que permita conocer qué acciones se realizaron sobre una solicitud, quién las realizó y en qué momento, lo que genera falta de transparencia, dificultad para resolver reclamos y problemas para identificar errores o malas prácticas dentro del sistema.

Ante esta situación, se vuelve necesario contar con un módulo de auditoría y monitoreo que registre de forma automática y confiable todos los eventos relevantes del sistema, así como los cambios realizados sobre la información crítica. Este módulo permitirá a los auditores, administradores y personal de soporte consultar una bitácora de eventos, aplicar filtros y búsquedas, y analizar la trazabilidad de los cambios realizados, contribuyendo así a mejorar el control, la seguridad y la confianza en la plataforma Campus360.

2. Justificación

En una plataforma universitaria como Campus360, donde múltiples usuarios interactúan constantemente realizando solicitudes, aprobaciones y actualizaciones de información, resulta fundamental contar con mecanismos que permitan controlar y supervisar dichas acciones. La ausencia de un sistema de auditoría dificulta la identificación de responsabilidades ante errores, reclamos o incidentes, y limita la capacidad de análisis sobre el comportamiento del sistema y sus usuarios.

3. Objetivo

Diseñar e implementar un módulo de Auditoría y Monitoreo que permita **r**egistrar, almacenar y consultar los eventos y cambios realizados en el sistema Campus360, asegurando la trazabilidad de las acciones efectuadas por los distintos usuarios.

4. Alcance de negocio

El módulo de Auditoría y Monitoreo se encarga de registrar y almacenar los eventos generados por los distintos módulos del sistema Campus360, permitiendo la trazabilidad de los cambios realizados. Asimismo, brinda a los usuarios autorizados la posibilidad de consultar la bitácora de eventos mediante filtros y búsquedas. El módulo no incluye la gestión de autenticación ni la ejecución de procesos operativos, ya que actúa únicamente como un servicio de control y monitoreo del sistema.

5. Actores del sistema

- **Auditor:** Responsable de garantizar el cumplimiento normativo de los servicios de Auditoría.
  Consulta la bitácora de eventos, aplica filtros y exporta los registros.
- **Administrador:** Encargado de la gestión operativa y del acceso al sistema según roles.
  Supervisa eventos del sistema.
- **Soporte TI:** Perfil técnico que utiliza el módulo como herramienta de diagnóstico para el mantenimiento y la resolución de fallos técnicos en la plataforma.
  Revisa registros y eventos ante incidencias.

6. Lista de requisitos

6.1. Requisitos Funcionales

Requisito	Nombre	Descripción
RF1	Registro de eventos	El sistema debería ser capaz de registrar automáticamente todos los eventos relevantes generados por otros módulos del sistema.
RF2	Registro de cambios	El sistema debería ser capaz de registrar las modificaciones realizadas sobre la información, mostrando explícitamente los valores anteriores y nuevos.
RF3	Consulta de bitácora	El sistema debería ser capaz de permitir a los usuarios autorizados visualizar la lista completa de eventos registrados en la bitácora.
RF4	Filtros de búsqueda	El sistema debería ser capaz de aplicar filtros específicos por fecha, usuario, tipo de acción y entidad para facilitar la localización de registros.
RF5	Control de acceso	El sistema debería ser capaz de restringir el acceso a la información de auditoría validando estrictamente el rol del usuario.
RF6	Historial por entidad	El sistema debería ser capaz de consultar de manera agrupada todos los eventos y cambios asociados a una entidad o registro específico del sistema.

RF7	Exportación de datos	El sistema debería ser capaz de exportar los registros de la bitácora a un archivo descargable para su posterior análisis externo.

6.2. Requisitos No Funcionales

Requisito	Nombre	Descripción
RNF1	Seguridad	El sistema debería ser capaz de garantizar un acceso restringido mediante la validación estricta de roles de usuario.
RNF2	Rendimiento	El sistema debería ser capaz de procesar y responder a las consultas en un tiempo menor a 2 segundos en el entorno local.
RNF3	Disponibilidad	El sistema debería ser accesible para los usuarios autorizados en todo momento para realizar consultas.
RNF4	Mantenibilidad	El sistema debería ser diseñado con un código organizado por capas que resulte fácil de extender y mantener.
RNF5	Usabilidad	El sistema debería ser intuitivo, proporcionando filtros y mecanismos de búsqueda que sean claros y fáciles de utilizar.

7. Backlog

ID	Historia de Usuario	Prioridad	Criterios de aceptación
HU1	Como auditor, quiero consultar la bitácora para revisar las acciones realizadas	Alta	Se muestra lista de eventos con fecha, usuario y acción
HU2	Como auditor, quiero filtrar eventos por fecha para analizar periodos	Alta	El sistema permite seleccionar rango de fechas
HU3	Como auditor, quiero ver el detalle de un evento para conocer qué cambió	Alta	Se muestra valor anterior y nuevo
HU4	Como administrador, quiero filtrar eventos por usuario para identificar responsables	Media	El sistema muestra solo eventos del usuario seleccionado
HU5	Como soporte TI, quiero consultar eventos por tipo de acción para analizar errores	Media	El sistema permite filtrar por acción

HU6	Como sistema, quiero registrar automáticamente los eventos generados por otros módulos	Alta	Cada acción relevante genera un registro
HU7	Como auditor, quiero exportar la bitácora para análisis externo	Baja	Se puede descargar archivo con eventos
HU8	Como auditor, quiero consultar historial de cambios de una entidad	Alta	El sistema muestra todos los cambios asociados
HU9	Como administrador, quiero asegurar que solo usuarios autorizados accedan a auditoría	Alta	El sistema valida roles antes de mostrar información

8. Reglas de negocio

Regla	Nombre	Descripción
RN1	Registro obligatorio	El sistema debería ser capaz de generar un registro de auditoría de forma automática ante cada acción relevante o cambio de estado realizado por los usuarios.
RN2	Inmutabilidad de los registros	El sistema debería ser diseñado de modo que los registros de auditoría almacenados no puedan ser modificados ni eliminados bajo ninguna circunstancia.
RN3	Control de acceso por roles	El sistema debería ser capaz de restringir el acceso a la información de auditoría únicamente a los usuarios que posean los roles de Auditor, Administrador o Soporte TI.
RN4	Información mínima del registro	El sistema debería ser capaz de asegurar que cada registro contenga al menos la fecha y hora, la identificación del usuario, el tipo de acción y la entidad afectada.
RN5	Trazabilidad de cambios	El sistema debería ser capaz de almacenar y mostrar tanto el valor anterior como el valor nuevo de cualquier dato que haya sido modificado.
RN6	Consulta filtrada	El sistema debería ser capaz de permitir la búsqueda de información aplicando filtros por rango de fechas, usuario responsable, tipo de acción o entidad específica.
RN7	Exportación de información	El sistema debería ser capaz de permitir la exportación de la bitácora de auditoría únicamente a aquellos usuarios que cuenten con los permisos explícitos para dicha acción.

RN8	Integridad de la información	El sistema debería ser capaz de garantizar que los datos de auditoría se mantengan íntegros y protegidos contra fallos del sistema o intentos de manipulación externa.

9. Estilo arquitectónico

El módulo de Auditoría y Monitoreo adopta una arquitectura híbrida que combina el estilo Cliente–Servidor con servicios REST para la comunicación con otros módulos del sistema Campus360, y una arquitectura en capas para la organización interna del servidor.

A nivel de integración, el módulo expone y consume servicios mediante una API REST, lo que permite recibir eventos generados por otros módulos del sistema y ofrecer mecanismos de consulta de la bitácora de auditoría de forma desacoplada. Este enfoque facilita la interoperabilidad, la escalabilidad y la integración con los distintos componentes de la plataforma.

A nivel interno, el servidor se organiza en capas, separando claramente las responsabilidades en:

- **Capa de Presentación**, encargada de atender las solicitudes HTTP y devolver las respuestas al cliente.

- **Capa de Aplicación**, responsable de coordinar los casos de uso, como registrar eventos, consultar la bitácora o aplicar filtros.

- **Capa de Dominio**, donde se define la lógica principal del sistema, incluyendo las reglas de negocio relacionadas con la trazabilidad, la integridad y la inmutabilidad de los registros de auditoría.

- **Capa de Infraestructura**, encargada del acceso a la base de datos y de los mecanismos técnicos de persistencia y almacenamiento.

Esta separación permite aislar la lógica del negocio de los detalles técnicos, facilitando el mantenimiento, la extensibilidad del sistema y la realización de pruebas unitarias. Asimismo, la arquitectura resulta adecuada para aplicaciones web como Campus360, donde múltiples usuarios acceden simultáneamente al sistema y requieren consultas confiables y seguras de la información.

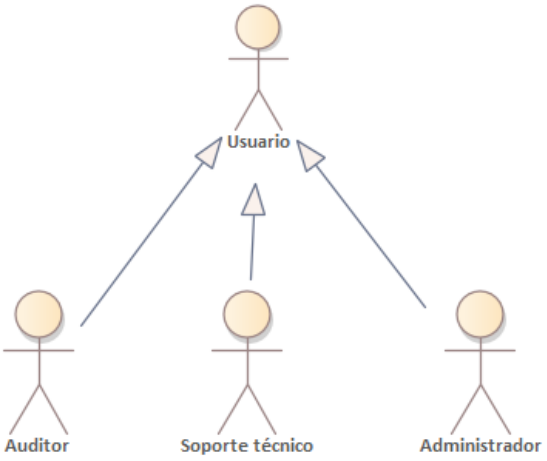
10. Arquitectura 4 +1

10.1. Casos de uso

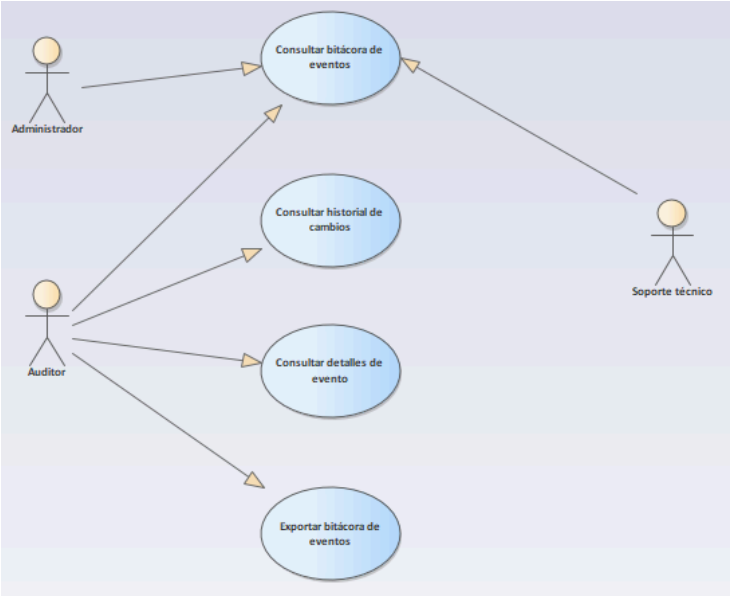
10.1.1. Casos de uso

- Consultar bitácora de eventos
- Consultar historial de cambios
- Consultar detalles de evento
- Exportar bitácora de eventos

10.1.2. Diagrama de Actores



10.2. Diagrama de casos de Uso



10.3. Descripción de casos de uso

Caso 1	Consultar bitácora de eventos
Actores	Auditor, Administrador, Soporte TI
Objetivo	Permitir a los usuarios autorizados visualizar la lista de eventos registrados en el sistema
Precondiciones	
- El usuario ha iniciado sesión en el sistema. - El usuario posee un rol autorizado para acceder al módulo de auditoría. - Existen eventos registrados en la bitácora o el sistema está disponible para consultarlos.	
Flujo principal	
1. El usuario accede al módulo de auditoría. 2. El sistema muestra la pantalla de consulta de bitácora. 3. El usuario ingresa criterios de búsqueda (fecha, usuario, acción o entidad). 4. El usuario solicita la consulta. 5. El sistema valida los filtros ingresados. 6. El sistema consulta los eventos registrados. 7. El sistema muestra la lista de eventos encontrados.	
Flujos Alternos	
6a. El sistema no obtiene eventos registrados con los filtros determinados. Se muestra un mensaje al usuario.	
Poscondiciones	
La bitácora de eventos se muestra en pantalla según los filtros aplicados.	
Reglas de Negocio	
RN3. Control de acceso por roles, solo los que tengan roles (Actores mencionados) deberán acceder a esta información. RN6. Consulta filtrada	

Datos Involucrados
Usuario: idUsuario, rol EventoAuditoria: idEvento, fecha ControladorAuditoria: listarEventos() ServicioAuditoria: consultarEventos() RepositorioAuditoria: obtenerEventos()

Caso 2	Consultar historial de cambios
Actores	Auditor, Administrador, Soporte TI
Objetivo	Permitir consultar el historial de cambios realizados sobre una entidad específica
Precondiciones	
- El usuario ha iniciado sesión. - El usuario tiene permisos para acceder al módulo de auditoría. - Existe al menos una entidad con registros de auditoría.	
Flujo principal	
1. El usuario accede al módulo de auditoría. 2. El usuario selecciona una entidad o registro. 3. El usuario solicita ver el historial de cambios. 4. El sistema busca los eventos asociados a la entidad. 5. El sistema obtiene el historial de cambios. 6. El sistema muestra la lista de cambios realizados, incluyendo fechas y usuarios.	
Flujos Alternos	
5a. El sistema no obtiene el historial de cambios, debido a que no existen cambios en esta entidad. El sistema muestra una lista vacía de cambios al usuario.	

Poscondiciones
- El historial de cambios de la entidad queda visible en pantalla. - No se altera la información almacenada.
Reglas de Negocio
RN2. Inmutabilidad de los registros RN3. Control de acceso por roles RN5. Trazabilidad de cambios RN6. Consulta filtrada
Datos Involucrados
Usuario: idUsuario, rol, nombre EventoAuditoria: idEvento, fecha Entidad: idEntidad, tipoEntidad Cambio: idCambio, valorAntiguo, valorNuevo ControladorAuditoria: verHistoriaEntidad() ServicioAuditoria: consultarHistorialEntidad() RepositorioAuditoria: obtenerEventosEntidad()

Caso 3	Consultar detalles de evento
Actores	Auditor, Administrador, Soporte TI
Objetivo	Permitir visualizar el detalle completo de un evento registrado en la bitácora.
Precondiciones	
- El usuario ha iniciado sesión. - El usuario tiene permisos para acceder al módulo de auditoría. - Existe al menos una entidad con registros de auditoría.	
Flujo principal	
1. El usuario consulta la bitácora de eventos. 2. El usuario selecciona un evento específico.	

3. El usuario solicita ver el detalle del evento. 4. El sistema obtiene la información del evento. 5. El sistema muestra el detalle, incluyendo valores anteriores y nuevos, usuario y fecha.
Flujos Alternos
4a. El sistema no obtiene la información del evento debido a un error. El sistema muestra un mensaje de error al usuario.
Poscondiciones
- El detalle completo del evento queda visible en pantalla. - No se modifica la información del sistema..
Reglas de Negocio
RN3. Control de acceso por roles RN4. Información mínima obligatoria RN5. Trazabilidad de cambios
Datos Involucrados
Usuario: idUsuario, rol EventoAuditoria: idEvento, fecha, accion, descripcion Cambio: idCambio, valorAntiguo, valorNuevo, campo ControladorAuditoria: verDetalleEvento() ServicioAuditoria: consultarDetalleEvento() RepositorioAuditoria: obtenerEventosId()

Caso 4	Exportar bitácora de evento
Actores	Auditor, Administrador, Soporte TI
Objetivo	Permitir exportar los registros de auditoría para su análisis externo.
Precondiciones	
- El usuario ha iniciado sesión. - El usuario tiene permisos para exportar información.	

- Existen eventos registrados o filtros aplicables.

Flujo principal

1. El usuario accede al módulo de auditoría.
2. El usuario aplica filtros si lo desea.
3. El usuario solicita exportar la bitácora.
4. El sistema obtiene los eventos correspondientes.
5. El sistema genera el archivo de exportación.
6. El sistema permite descargar el archivo al usuario.

Flujos Alternos

4a. El sistema no obtiene eventos debido a los filtros aplicados.
El sistema muestra un mensaje al usuario “No se encuentran coincidencias”.

Poscondiciones

- El archivo de auditoría es generado y descargado correctamente.
- Los registros del sistema permanecen sin modificaciones..

Reglas de Negocio

RN3. Control de acceso por roles
RN6. Consulta filtrada
RN7. Exportación de información
RN8. Integridad de la información

Datos Involucrados

Usuario: idUsuario, rol
EventoAuditoria: idEvento, fecha, accion, descripcion
ControladorAuditoria: listarEventos(), exportarBitacora()
ServicioAuditoria: consultarEventos(), exportarBitacora()
RepositorioAuditoria: guardarEvento()

10.4. Escenarios Significativos

Escenario significativo 1

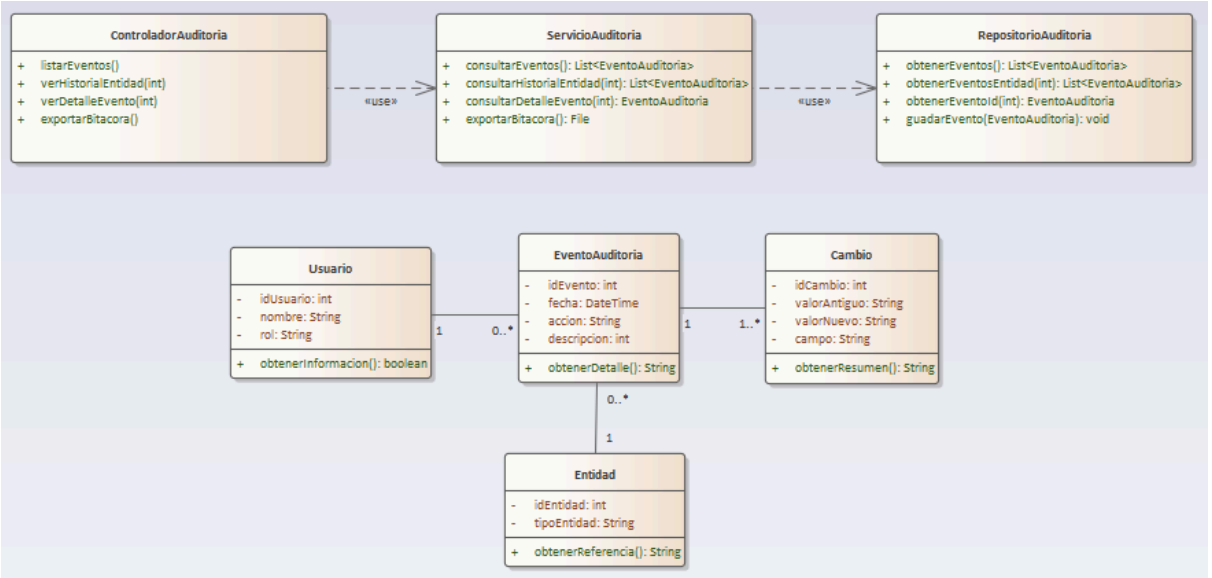
El Auditor ingresa al sistema Campus 360 y accede al módulo de auditoría con el objetivo de investigar un cambio inusual en las calificaciones de un curso. Consulta la bitácora de eventos para un rango de fechas específico, aplicando filtros por la entidad "Acta de Notas" y el tipo de acción "Modificar". El sistema procesa la búsqueda y muestra un listado de eventos coincidentes. El Auditor selecciona uno de los registros para ver el detalle. El sistema despliega la información completa, mostrando explícitamente que el valor anterior era "10" y el valor nuevo es "18", junto con el usuario responsable del cambio. Tras validar la inconsistencia, el Auditor solicita la exportación de la evidencia. El sistema genera un archivo con la información del evento, asegurando la integridad de los datos para el reporte oficial.

Escenario significativo 2

El Soporte TI recibe un reporte sobre una caída del sistema durante un proceso de matrícula e ingresa al módulo de monitoreo para diagnosticar la causa. Accede a la consulta de eventos y filtra la búsqueda por el tipo de acción "Error de Sistema" y el rango horario del incidente reportado. El sistema muestra una lista cronológica de fallos registrados. El Soporte TI selecciona el evento crítico y revisa los detalles técnicos almacenados. El sistema revela que el error fue desencadenado por una excepción de base de datos tras una acción masiva realizada por un usuario administrativo específico. Con esta información de trazabilidad, el Soporte TI identifica el origen del fallo, permitiendo iniciar las correcciones necesarias y notificar al área correspondiente.

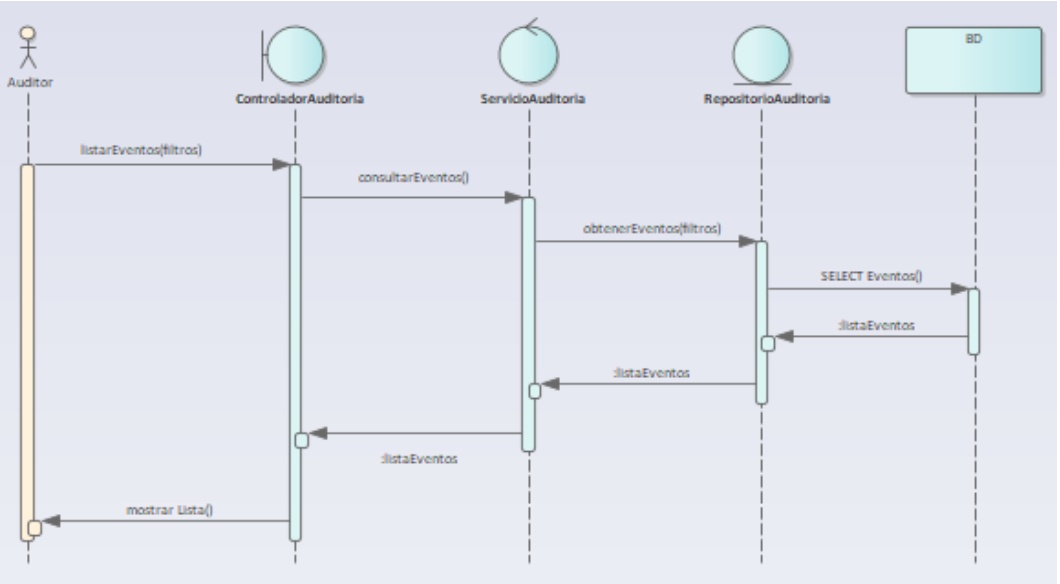
10.5. Vista lógica

10.5.1. Diagrama de clases

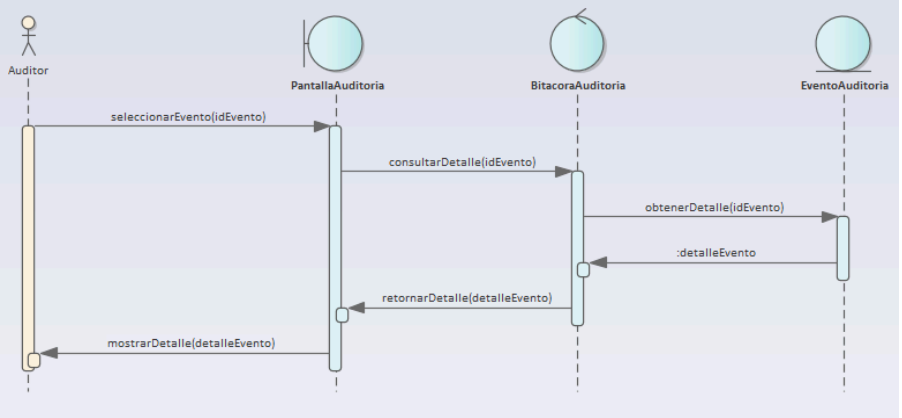


10.5.2. Diagrama de secuencia

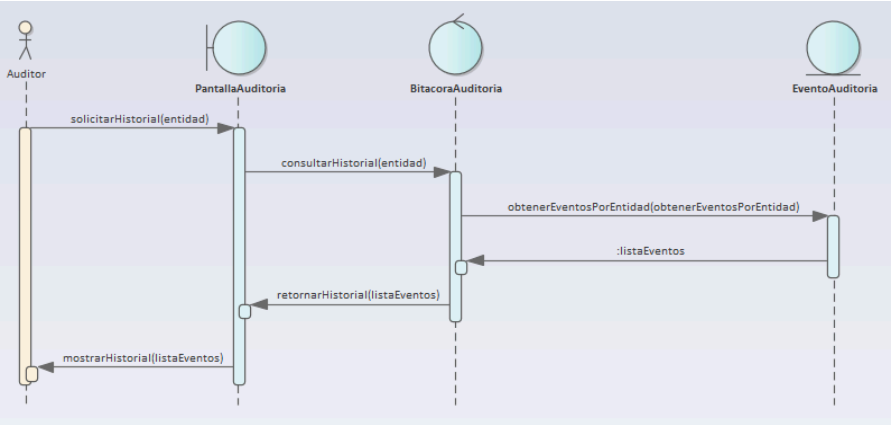
10.5.2.1. Consultar bitácora de eventos



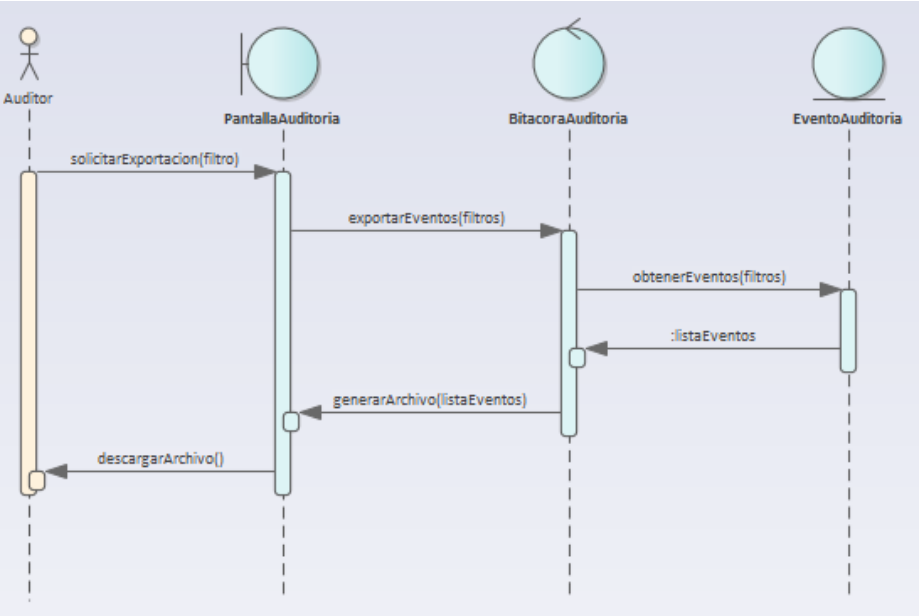
10.5.2.2. Consultar detalles de evento



10.5.2.3. Consultar historial de cambios

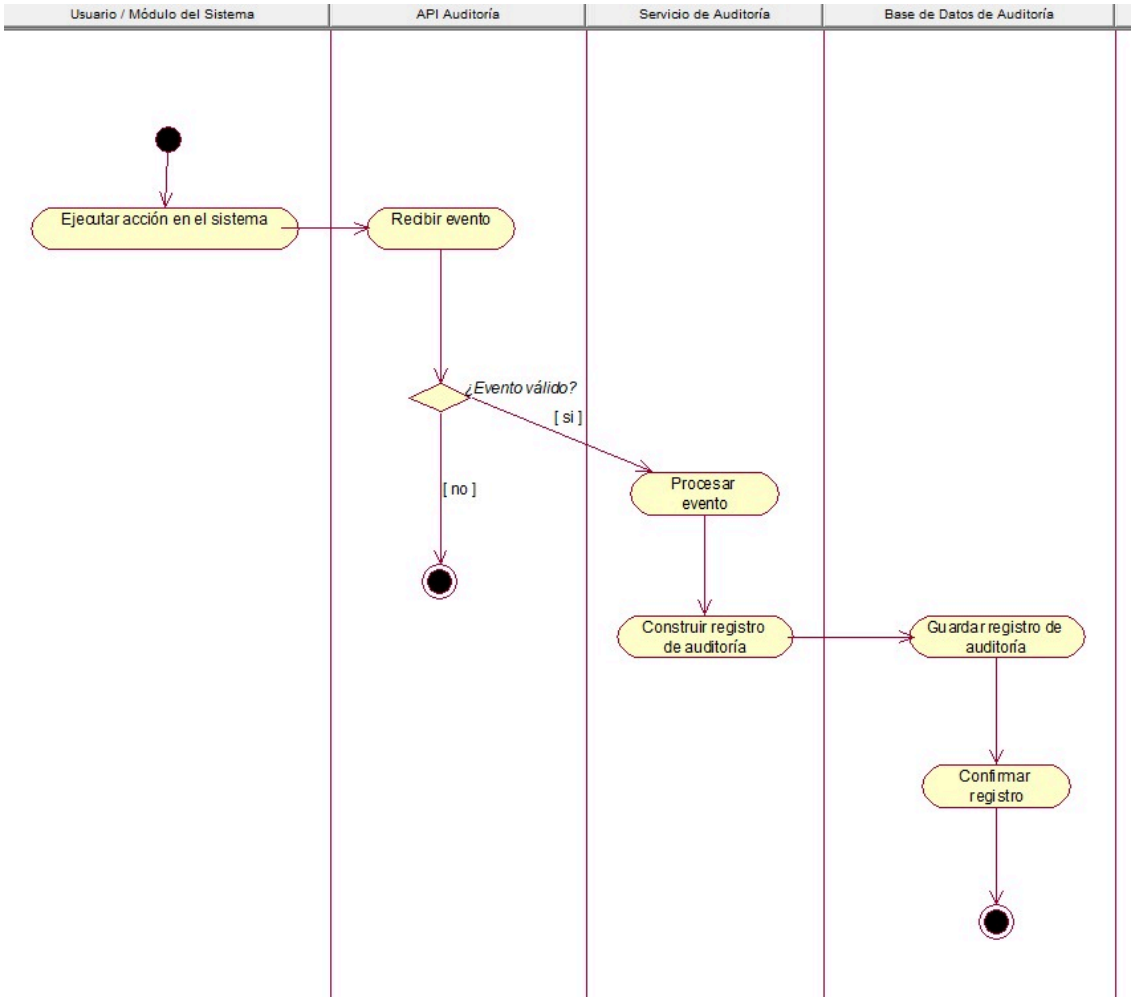


10.5.2.4. Exportar bitácora de eventos

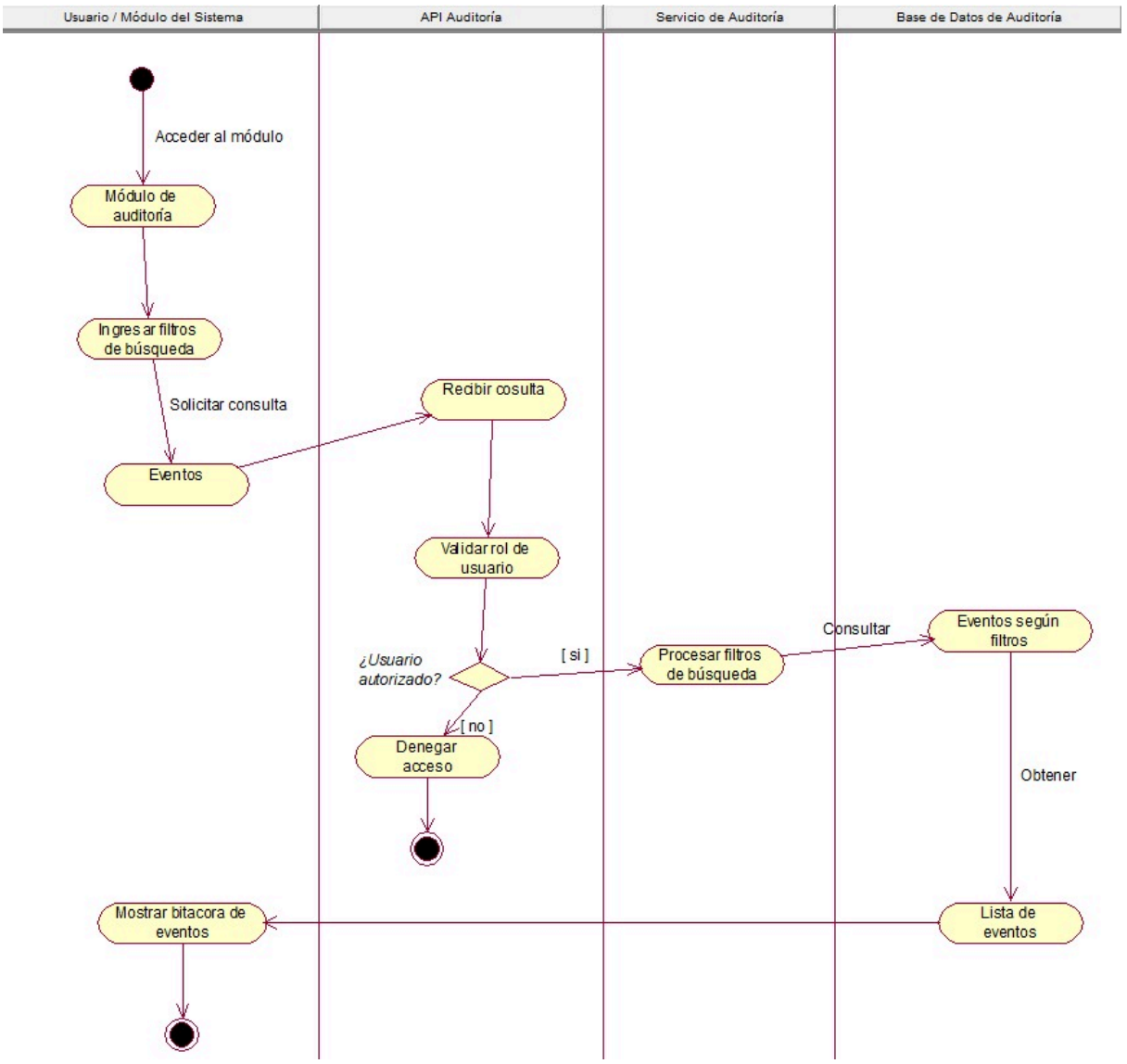


10.6. Vista de procesos

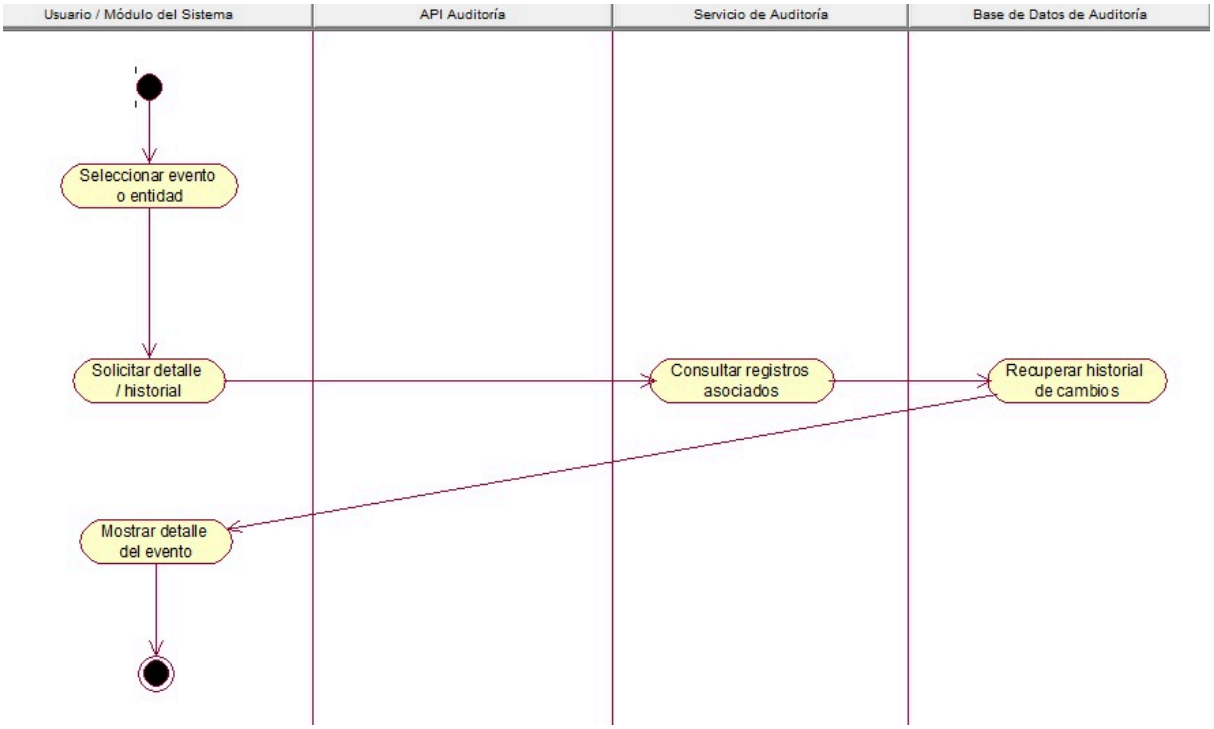
Flujo A: Registro automático de eventos



Flujo B: Consulta de bitácora de eventos



Flujo C: Ver detalle / historial de cambios



Flujo D: Exportar bitácora

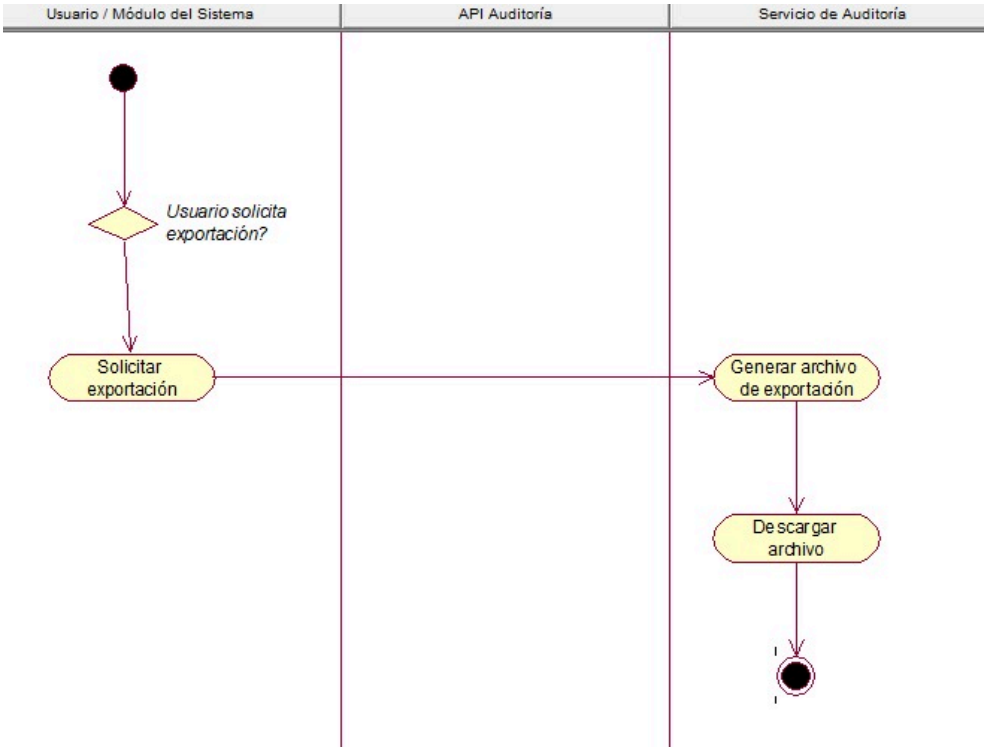
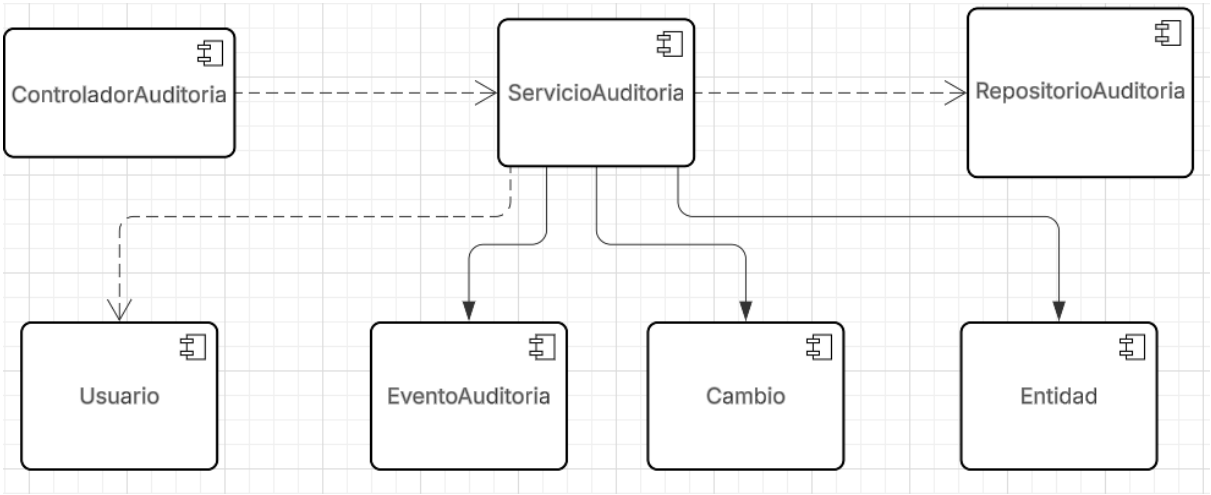


Figura 2. Diagrama de actividades del módulo de Auditoría y Monitoreo

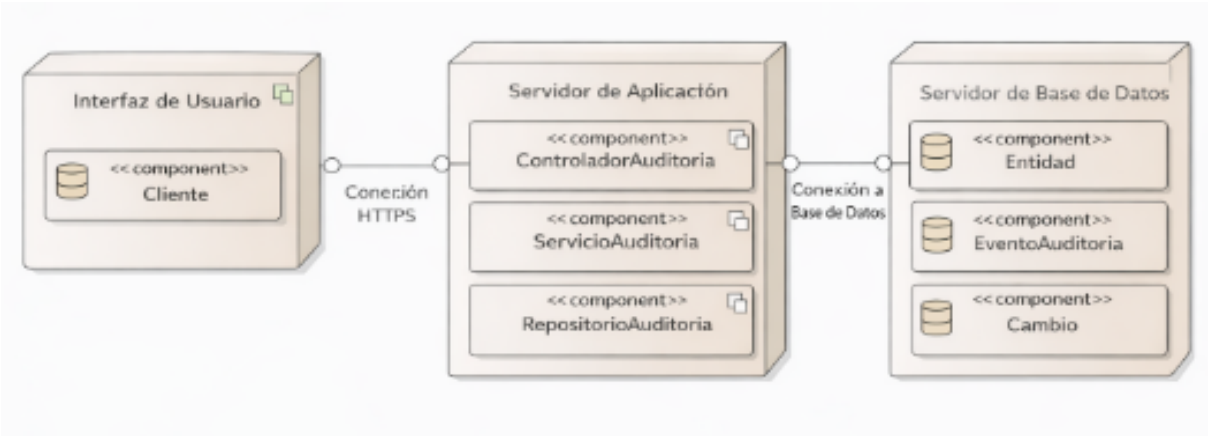
10.7. Vista de implementación

DIagrama de Componentes



10.8. Vista de despliegue

Diagrama de Despliegue



11. Diseño de Interfaces del Módulo de Auditoría

11.1. Objetivo del Diseño

El diseño de interfaces del módulo de Auditoría y Monitoreo tiene como objetivo proporcionar una experiencia clara, intuitiva y profesional que permita a los usuarios autorizados consultar eventos registrados en el sistema Campus360 de manera eficiente.
Se priorizó:

- Claridad visual
- Organización estructurada de información
- Uso de filtros intuitivos
- Visualización tabular ordenada
- Estética profesional tipo dashboard empresarial

11.2. Principios de Diseño Aplicados

Se aplicaron los siguientes principios:

- Consistencia visual en colores y tipografía
- Separación clara entre panel de navegación y contenido
- Uso de tabla estructurada para la bitácora
- Filtros agrupados en una sola sección superior
- Acciones visibles mediante botones

11.3. Estructura de la Interfaz

La interfaz principal del módulo se compone de:

- Panel lateral de navegación
- Título principal del módulo
- Sección de filtros
- Tabla de resultados
- Botón de exportación

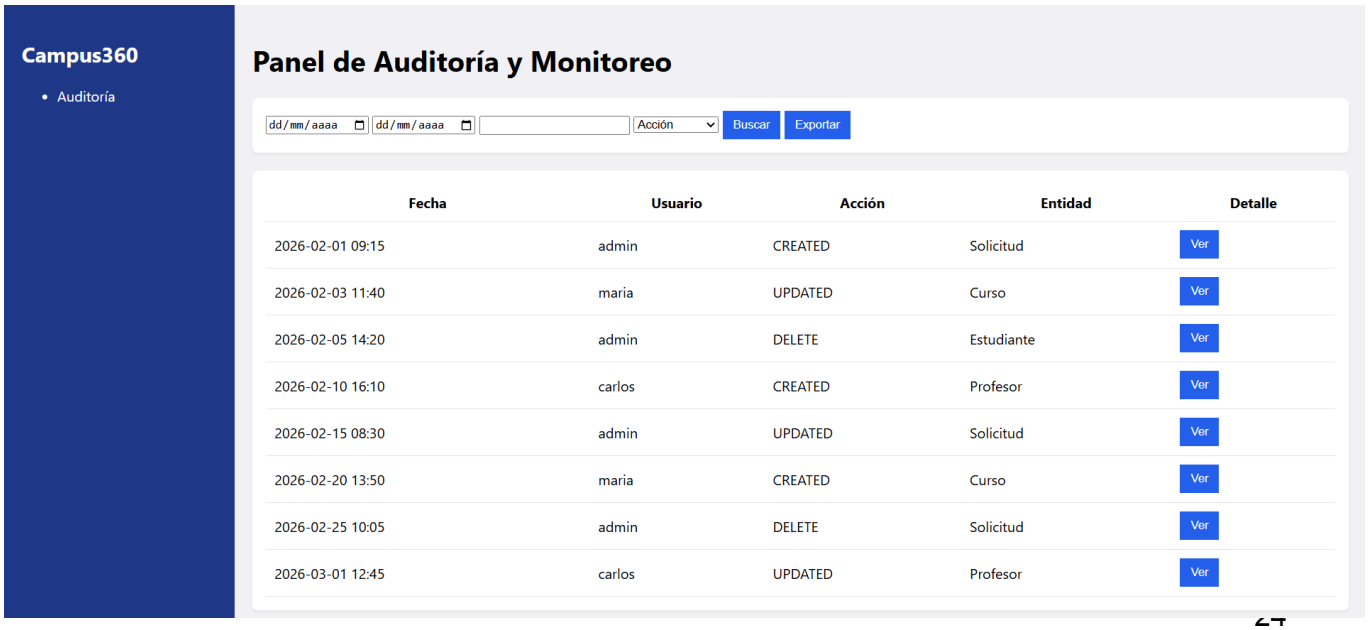


Figura 1. Vista general del módulo de Auditoría y Monitoreo.

11.4. Sección de Filtros

La sección de filtros permite realizar consultas dinámicas sobre la bitácora.

Incluye:

- Selector de fecha inicio
- Selector de fecha fin
- Campo de usuario
- Selector de acción
- Botón Buscar
- Botón Exportar

El uso de dos selectores de fecha permite realizar consultas por rango temporal, lo cual es fundamental en procesos de auditoría donde se requiere analizar eventos ocurridos dentro de un periodo específico.

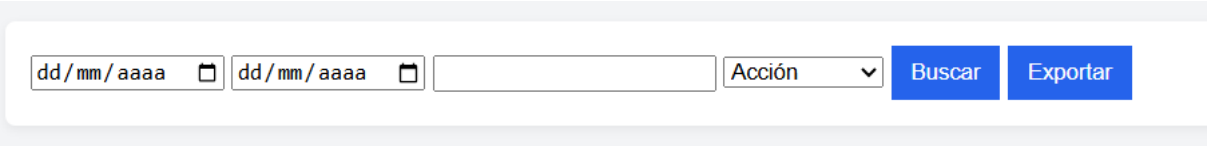


Figura 2. Sección de filtros para consulta de eventos.

11.5. Tabla de Resultados

La tabla muestra:

- Fecha y hora del evento
- Usuario responsable
- Acción realizada
- Entidad afectada
- Botón de detalle

La información se presenta en formato tabular para facilitar la lectura y análisis.

Fecha	Usuario	Acción	Entidad	Detalle
2026-02-01 09:15	admin	CREATED	Solicitud	Ver
2026-02-03 11:40	maria	UPDATED	Curso	Ver
2026-02-05 14:20	admin	DELETE	Estudiante	Ver
2026-02-10 16:10	carlos	CREATED	Profesor	Ver
2026-02-15 08:30	admin	UPDATED	Solicitud	Ver
2026-02-20 13:50	maria	CREATED	Curso	Ver
2026-02-25 10:05	admin	DELETE	Solicitud	Ver
2026-03-01 12:45	carlos	UPDATED	Profesor	Ver

Figura 3. Tabla de resultados de auditoría.

11.6. Decisiones Técnicas de Interfaz

- Fuente profesional Inter para apariencia corporativa.
- Diseño responsive básico.
- Uso de JavaScript para filtrado dinámico.
- Validación de fechas en frontend.
- Estructura modular del código.

12. Prototipo Mínimo Funcional (MVP)

12.1. Descripción del MVP

El prototipo mínimo funcional implementa las funcionalidades esenciales del módulo:

- Registro y consulta de eventos
- Filtrado por rango de fechas
- Filtrado por usuario y acción
- Visualización de detalle
- Simulación de exportación

El sistema se implementó con arquitectura Cliente-Servidor utilizando:

Frontend: HTML, CSS, JavaScript
Backend: Spring Boot (API REST)

12.2. Definición de Endpoints

Método	Endpoint	Descripción	Rol Requerido	Errores
GET	/api/v1/audit/events	Listar eventos con filtros	Auditor, Admin	401, 403
GET	/api/v1/audit/events/{id}	Ver detalle de evento	Auditor, Admin	404
GET	/api/v1/audit/history/{entityId}	Historial de cambios	Auditor	404
GET	/api/v1/audit/export	Exportar eventos	Auditor	404

12.3. Extracto de Código – Filtrado por Fecha (Frontend)

```
if (startDateValue) {
    const startDate = new Date(startDateValue);
    if (fechaEvento < startDate) {
        cumpleFecha = false;
    }
}
```

Explicación:

Este fragmento valida que el evento ocurra después de la fecha inicial seleccionada.

12.4. Extracto de Código – Endpoint REST (Backend)

```
@GetMapping("/api/v1/audit/events")
public List<EventoAuditoria> listarEventos(
    @RequestParam(required = false) String startDate,
    @RequestParam(required = false) String endDate,
    @RequestParam(required = false) String usuario,
    @RequestParam(required = false) String accion) {

  return servicioAuditoria.consultarEventos(startDate, endDate, usuario, accion);
}
```

Explicación:

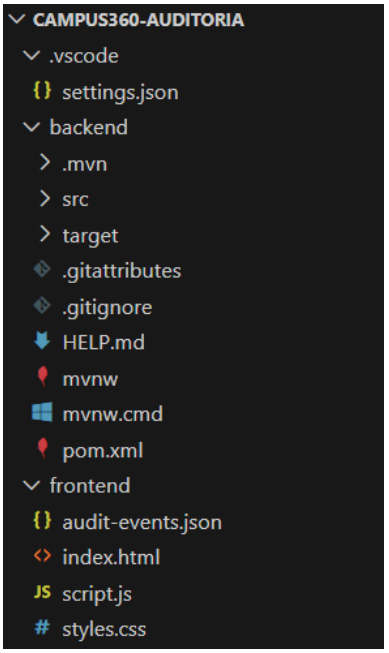
Permite consultar eventos aplicando filtros dinámicos enviados desde el frontend.

12.5. Arquitectura del Prototipo

El MVP sigue arquitectura en capas:

- Controlador
- Servicio
- Repositorio
- Base de datos

La comunicación se realiza mediante API REST.



13. Pruebas

Edpoints

Pruebas unitarias

1. **Test: obtenerTodos()**

   Este test verifica que el método obtenerTodos() devuelva correctamente todos los eventos de auditoría almacenados en el repositorio.

   Cómo funciona

   - Se crea un evento de auditoría de prueba.
   - Se simula el repositorio para que devuelva ese evento.
   - Se ejecuta el método obtenerTodos().
   - Se verifica que la lista tenga un elemento.
   - Se verifica que la acción del evento sea correcta.

```
@Test
void testObtenerTodos() {
    EventoAuditoria evento = new EventoAuditoria();
    evento.setAccion("CREAR");

    when(eventoRepo.findAll()).thenReturn(List.of(evento));

    List<EventoAuditoria> resultado = service.obtenerTodos();

    assertEquals(1, resultado.size());
    assertEquals("CREAR", resultado.get(0).getAccion());
}
```

2. **Test: obtenerHistorial()**

   Este test verifica que el método obtenerHistorial(idEntidad) obtenga los eventos asociados a una entidad específica.

   Cómo funciona

   - Se crea un evento de auditoría.
   - Se simula el repositorio para devolver eventos de la entidad con ID 1.
   - Se ejecuta el método obtenerHistorial(1L).
   - Se verifica que se obtenga un evento.

   Qué valida

- Que el servicio consulte por ID de entidad.
- Que el repositorio sea llamado correctamente.
- Que los eventos se devuelvan correctamente.

```
@Test
void testObtenerHistorial() {
    EventoAuditoria evento = new EventoAuditoria();

    when(eventoRepo.findByEntidad_IdEntidad(1L))
            .thenReturn(List.of(evento));

    List<EventoAuditoria> resultado =
            service.obtenerHistorial(1L);

    assertEquals(1, resultado.size());
}
```

3. **Test: guardarEvento() Cprrecto**

Este test verifica que el método guardarEvento() guarde correctamente un evento cuando el usuario y la entidad existen.

Cómo funciona

- Se crea un usuario con ID 1.
- Se crea una entidad con ID 1.
- Se crea un evento con ese usuario y entidad.
- Se simulan los repositorios para devolver usuario y entidad.
- Se simula el guardado del evento.
- Se ejecuta el método guardarEvento().
- Se verifica que el evento se guarde.

Qué valida

- Que el usuario exista.
- Que la entidad exista.
- Que el evento se guarde.
- Que el repositorio save sea llamado.

```java
@Test
void testGuardarEventoCorrecto() {

    Usuario usuario = new Usuario();
    usuario.setIdUsuario(1L);

    Entidad entidad = new Entidad();
    entidad.setIdEntidad(1L);

    EventoAuditoria evento = new EventoAuditoria();
    evento.setUsuario(usuario);
    evento.setEntidad(entidad);

    when(usuarioRepo.findById(1L))
            .thenReturn(Optional.of(usuario));

    when(entidadRepo.findById(1L))
            .thenReturn(Optional.of(entidad));

    when(eventoRepo.save(any(EventoAuditoria.class)))
            .thenReturn(evento);

    EventoAuditoria resultado = service.guardarEvento(evento);

    assertNotNull(resultado);
    verify(eventoRepo, times(1)).save(evento);
}
```

4. **Test: guardarEvento() usuario no existe**

   Este test verifica que el método guardarEvento() lance una excepción cuando el usuario no existe.

   Cómo funciona

   - Se crea un usuario con ID inexistente.
   - Se simula el repositorio para que no encuentre el usuario.
   - Se ejecuta el método guardarEvento.
   - Se espera una excepción.

   Qué valida

   - Que el sistema valide la existencia del usuario.

- Que se lance un error si no existe.
- Que el evento no se guarde.

```
@Test
void testGuardarEventoUsuarioNoExiste() {

    Usuario usuario = new Usuario();
    usuario.setIdUsuario(99L);

    EventoAuditoria evento = new EventoAuditoria();
    evento.setUsuario(usuario);
    evento.setEntidad(new Entidad());

    when(usuarioRepo.findById(99L))
            .thenReturn(Optional.empty());

    RuntimeException ex = assertThrows(RuntimeException.class,
            () -> service.guardarEvento(evento));

    assertEquals("Usuario no existe", ex.getMessage());
}
```

5. **Test: registrarCambioEstado()**

   Este test verifica que el método registrarCambioEstado() registre correctamente un cambio de estado en una entidad.

   Cómo funciona

   - Se crea un usuario.
   - Se crea una entidad.
   - Se simulan los repositorios.
   - Se ejecuta el método registrarCambioEstado.
   - Se verifica que el evento tenga acción CAMBIO_ESTADO.
   - Se verifica que exista un cambio.
   - Se verifica el nuevo estado.

   Qué valida

   - Que se cree un evento de auditoría.

- Que la acción sea correcta.
- Que se registre el cambio.
- Que el valor nuevo sea correcto.

```
@Test
void testRegistrarCambioEstado() {

    Usuario usuario = new Usuario();
    usuario.setIdUsuario(1L);

    Entidad entidad = new Entidad();
    entidad.setIdEntidad(1L);

    when(usuarioRepo.findById(1L))
            .thenReturn(Optional.of(usuario));

    when(entidadRepo.findById(1L))
            .thenReturn(Optional.of(entidad));

    when(eventoRepo.save(any(EventoAuditoria.class)))
            .thenAnswer(invocation -> invocation.getArgument(0));

    EventoAuditoria resultado =
            service.registrarCambioEstado(1L, "APROBADO", 1L);

    assertEquals("CAMBIO_ESTADO", resultado.getAccion());
    assertEquals(1, resultado.getCambios().size());
    assertEquals("APROBADO",
            resultado.getCambios().get(0).getValorNuevo());
}
```

6. **Test: obtenerEventosParaExportar()**

Este test verifica que el método obtenerEventosParaExportar() convierta correctamente los eventos en objetos DTO para exportación.

Cómo funciona

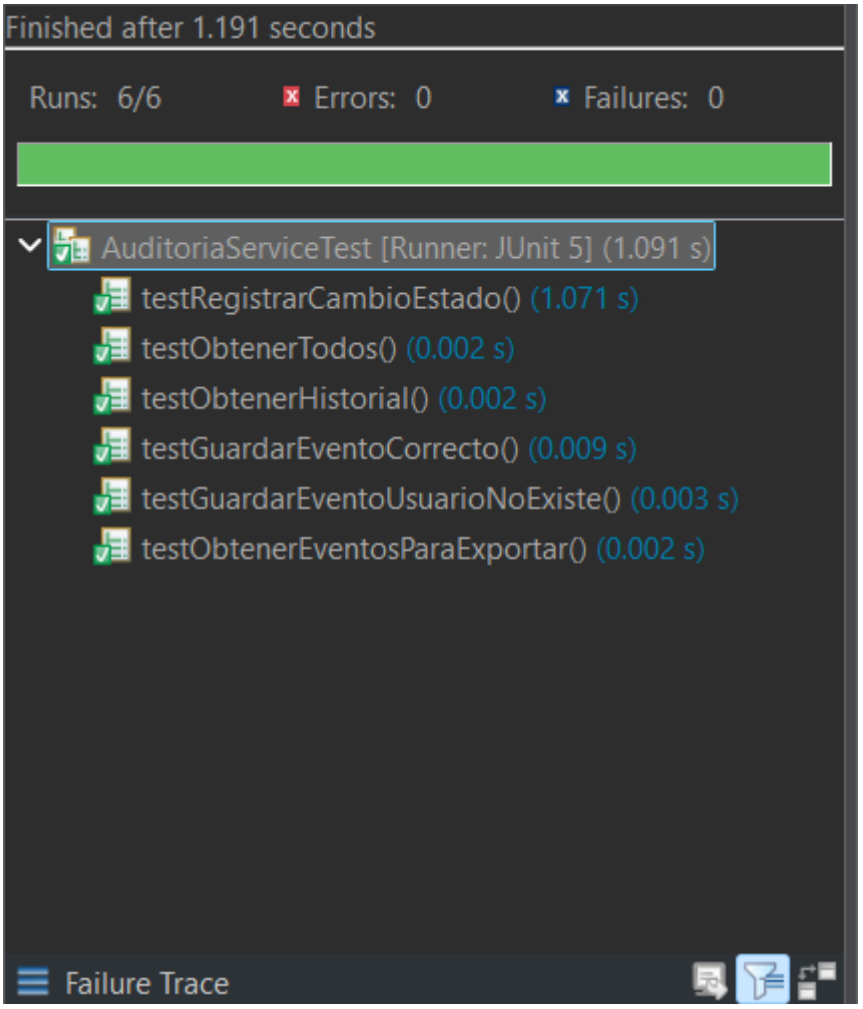
- Se crea un usuario.
- Se crea una entidad.
- Se crea un evento.
- Se simula el repositorio.

- Se ejecuta el método.
- Se verifica el resultado.

Qué valida

- Que los eventos se obtengan.
- Que se conviertan a DTO.
- Que los datos sean correctos.

Ejecución



Prueba de Caja blanca

Prueba de Caja negra

Caso	Entrada	Acción	Resultado esperado	Resultado obtenido
1	Crear evento	POST auditoria	Evento guardado	OK
2	Listar eventos	GET auditoria	Lista eventos	OK
3	Historial	GET historial/1	Cambios de entidad	OK
4	Usuario inválido	POST auditoria	Error	OK
5	Exportar	GET exportar	Descarga Excel	OK
6	Filtrar usuario	Frontend	Solo sus eventos	OK
7	Entidad inválida	POST auditoria	Error	OK
8	Sin eventos	GET auditoria	Lista vacía	OK